

Chapter 38

Unsupervised Learning

Run k-means by hand and in scikit-learn, and choose k with the elbow, the silhouette, stability, and business sense · profile and name clusters so a sales team can act on them · use hierarchical clustering and read a dendrogram

Approved chapter · v1

Contents

Chapter 38. Unsupervised Learning

Why this matters

In plain English

38.0 The accounts, prepared

38.1 k-means

38.2 Choosing k, and whether the clusters are real

38.3 Profiling and naming clusters

38.4 Hierarchical clustering

38.5 DBSCAN

38.6 Seeing many dimensions: PCA, t-SNE, UMAP

38.7 Anomaly detection

38.8 Judging unsupervised results

38.9 Market basket analysis

Common mistakes and how to spot them

In the real world: the four segments that already existed

Tools

The project: customer segmentation and a product-bundle analysis

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 38. Unsupervised Learning

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: run k-means by hand and in scikit-learn, and choose k with the elbow, the silhouette, stability, and business sense · profile and name clusters so a sales team can act on them · use hierarchical clustering and read a dendrogram · use DBSCAN, and know when density-based clustering suits a problem · reduce dimensions with PCA, t-SNE, and UMAP, and read their pictures without over-reading them · find anomalies with an isolation forest · run market basket analysis with support, confidence, and lift, by hand and with Apriori · judge unsupervised results, which have no test set to fall back on.

Before you start: Chapter 35 (distance, variance, PCA), Chapter 36 (scaling and pipelines), and Chapter 37 (the customer accounts dataset). No target variable is needed anywhere in this chapter.

Time needed: 8–12 hours over one to two weeks.

Tools: Python 3 with scikit-learn and SciPy, plus `umap-learn` and `mlxtend` (both free).

Practice data: Riverstone's **customer accounts** (5,000 rows, from Chapter 37) and a new **order baskets** dataset: 33,931 orders and 92,359 order lines from 2025 across a 24-product catalog, built by `companion/generate_riverstone_baskets.py`. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Everything in Chapters 36 and 37 needed a target: won or lost, churned or stayed. Most business data has no target. Nobody labeled Riverstone's 5,000 accounts as "type A" or "type B", and nobody wrote down which products belong together. **Unsupervised learning** finds structure in data that has no answer key.

It shows up constantly:

- "Can we group our customers so each group gets a different sales approach?" Clustering.
- "Which products should we bundle or recommend together?" Market basket analysis.
- "Which accounts look unusual enough to check by hand?" Anomaly detection.
- "We have 40 columns. Can we see them on one chart?" Dimensionality reduction.

It also has a trap that supervised learning doesn't. With no answer key, **there's no score that proves you're right**. Any dataset can be split into four groups; whether those groups mean anything is a separate question. That question, how to judge an unsupervised result, is the thread running through this chapter.

In plain English

Think of a new sales manager walking into Riverstone's warehouse office with a list of 5,000 customers and no notes.

Nobody can tell them which customers are important, so they start sorting. They put the big frequent buyers in one pile, the small occasional ones in another, and the ones who haven't ordered in months in a third. That's **clustering**: grouping by similarity, with nobody saying what the groups should be.

How many piles? Two is too crude, twenty is unusable. They try a few and pick the number where each pile can be described in a sentence and treated differently. That's **choosing k**.

Some customers fit no pile: one placed a single enormous order and vanished. They set those aside to look at individually. That's **anomaly detection**.

Then they look at the order book and notice that crates and crate lids keep appearing together. That's **market basket analysis**.

Finally, to explain it all to the sales head, they draw one chart with customers as dots, arranged so similar customers sit close together. That's **dimensionality reduction**.

38.0 The accounts, prepared

Clustering measures distance, so scaling is not optional (Chapter 35). Revenue is also skewed, so it goes in as a log. Only descriptive features are used: no `churned_2025`, because that's a target, and unsupervised methods don't get one. Later the churn column becomes a way to **check** the result from the outside.

```

import warnings

import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=UserWarning)

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["log_revenue"] = np.log(accounts["revenue_2024"])
accounts["late_payment_days"] = accounts["late_payment_days"].fillna(
    accounts["late_payment_days"].median()
)

FEATURES = [
    "log_revenue",
    "orders_2024",
    "categories_bought",
    "avg_discount_pct",
    "late_payment_days",
    "days_since_last_order",
    "tenure_months",
]
X = StandardScaler().fit_transform(accounts[FEATURES])
print(X.shape, "accounts x features")
print(accounts[FEATURES].describe().loc[["mean", "std"]].round(2).to_string())
print(
    "after scaling, every column has mean 0 and standard deviation 1:",
    np.allclose(X.mean(axis=0), 0),
    np.allclose(X.std(axis=0), 1),
)

```

```

(5000, 7) accounts x features
   log_revenue  orders_2024  categories_bought  avg_discount_pct  late_payment_days
days_since_last_order  tenure_months
mean           12.34        16.34              1.93             5.47             14.53
59.86           30.71
std            1.09         17.32              1.08             2.70             10.48
79.28           20.54
after scaling, every column has mean 0 and standard deviation 1: True True

```

How it works: `StandardScaler` subtracts each column's mean and divides by its standard deviation. After scaling, one standard deviation of revenue counts exactly as much as one standard deviation of order count, which is what "similar accounts" should mean. Note the raw standard deviations: orders vary by about 17 and log revenue by about 1. Without scaling, order count and recency would drown out everything else.

38.1 k-means

How it works

k-means splits rows into k groups, each represented by a **centroid**, the average of its members. It repeats two steps until nothing changes:

1. **Assign:** put every row in the cluster whose centroid is nearest (Euclidean distance).

2. **Update:** move each centroid to the average of the rows now assigned to it.

It minimizes **inertia**: the total squared distance from each row to its own centroid. Inertia always falls as k rises, which matters for choosing k .

By hand, on six accounts

Take six accounts described by two scaled features, and start with two centroids at the first and last account:

Account	Feature 1	Feature 2
A	1.0	2.0
B	1.5	1.8
C	5.0	8.0
D	8.0	8.0
E	1.0	0.6
F	9.0	11.0

Step 1, assign. Account C is $\sqrt{((5-1)^2 + (8-2)^2)} = \sqrt{52} = 7.21$ from centroid 1 and $\sqrt{((5-9)^2 + (8-11)^2)} = \sqrt{25} = 5.00$ from centroid 2, so it joins cluster 2. Doing the same for the rest gives A, B, E in cluster 1 and C, D, F in cluster 2.

Step 1, update. Cluster 1's new centroid is the average of A, B, E: $((1.0 + 1.5 + 1.0) \div 3, (2.0 + 1.8 + 0.6) \div 3) = (1.17, 1.47)$. Cluster 2's is $((5 + 8 + 9) \div 3, (8 + 8 + 11) \div 3) = (7.33, 9.00)$.

Step 2. Reassign with the new centroids: nothing changes, so the centroids stay put and the algorithm stops.

```
small = np.array(
    [[1.0, 2.0], [1.5, 1.8], [5.0, 8.0], [8.0, 8.0], [1.0, 0.6], [9.0, 11.0]]
)
centres = np.array([[1.0, 2.0], [9.0, 11.0]]) # start: first and last account

for step in range(1, 4):
    distances = np.linalg.norm(small[:, None, :] - centres[None, :, :], axis=2)
    labels = distances.argmin(axis=1)
    inertia = (distances.min(axis=1) ** 2).sum()
    print(f"step {step}: labels {labels} inertia {inertia:.2f}")
    print(f"centres {centres.round(2).tolist()}")
    new_centres = np.array([small[labels == c].mean(axis=0) for c in range(2)])
    if np.allclose(new_centres, centres):
        print("centres stopped moving: done")
        break
    centres = new_centres
```

```
step 1: labels [0 0 1 1 0 1] inertia 37.25
centres [[1.0, 2.0], [9.0, 11.0]]
step 2: labels [0 0 1 1 0 1] inertia 15.98
centres [[1.17, 1.47], [7.33, 9.0]]
centres stopped moving: done
```

Reading it. Inertia fell from 37.25 to 15.98 in one move, and the second pass changed nothing. Check the final inertia by hand for cluster 1: A is $(1.0 - 1.17, 2.0 - 1.47)$ from its centroid, a squared distance of $0.0289 + 0.2809 = 0.3098$; B gives $0.1089 + 0.1089 = 0.2178$; E gives $0.0289 + 0.7569 = 0.7858$. Cluster 1 contributes 1.313, and cluster 2 the remaining 14.67. ✓

```
from sklearn.cluster import KMeans

check = KMeans(n_clusters=2, n_init=10, random_state=38).fit(small)
print(
    "scikit-learn centres:", np.sort(check.cluster_centers_, axis=0).round(2).tolist()
)
print("labels:", check.labels_, " inertia:", round(check.inertia_, 2))
```

```
scikit-learn centres: [[1.17, 1.47], [7.33, 9.0]]
labels: [1 1 0 0 1 0] inertia: 15.98
```

scikit-learn finds the same centroids. Its cluster **numbers** are different (0 and 1 are swapped), which is normal: cluster labels are arbitrary names, not values. Never compare labels between two runs directly; compare the groupings (section 38.2 shows how).

Key settings: `n_clusters` (k), and `n_init`, the number of random starts. k -means can land in a poor solution depending on where the centroids begin, so scikit-learn runs it several times and keeps the best. `random_state` fixes the starts so results are repeatable.

Watch out: k -means assumes round, similar-sized, similar-density clusters. It draws straight boundaries halfway between centroids. Long thin groups, rings, and groups of very different sizes defeat it; that's where DBSCAN (section 38.4) and hierarchical clustering (section 38.3) help.

38.2 Choosing k , and whether the clusters are real

Three ways to choose, none decisive

The elbow. Plot inertia against k . It always falls; look for the bend where extra clusters stop buying much. It's often ambiguous, and this dataset is a fair example.

The silhouette. For each row: a is its average distance to its own cluster's members, b is its average distance to the nearest other cluster's members, and its silhouette is $(b - a) \div \max(a, b)$. It runs from -1 (in the wrong cluster) through 0 (on the boundary) to 1 (snugly inside). The average over all rows scores the whole clustering.

```
from sklearn.metrics import silhouette_score

print(" k    inertia    silhouette")
for k in range(2, 9):
    model = KMeans(n_clusters=k, n_init=10, random_state=38).fit(X)
    score = silhouette_score(X, model.labels_, sample_size=2000, random_state=38)
    print(f"{k:>2}    {model.inertia_:>7,.0f}    {score:>10.3f}")
```

k	inertia	silhouette
2	25,503	0.277
3	22,142	0.232
4	19,744	0.203
5	17,825	0.204
6	16,086	0.210
7	14,887	0.195
8	14,175	0.176

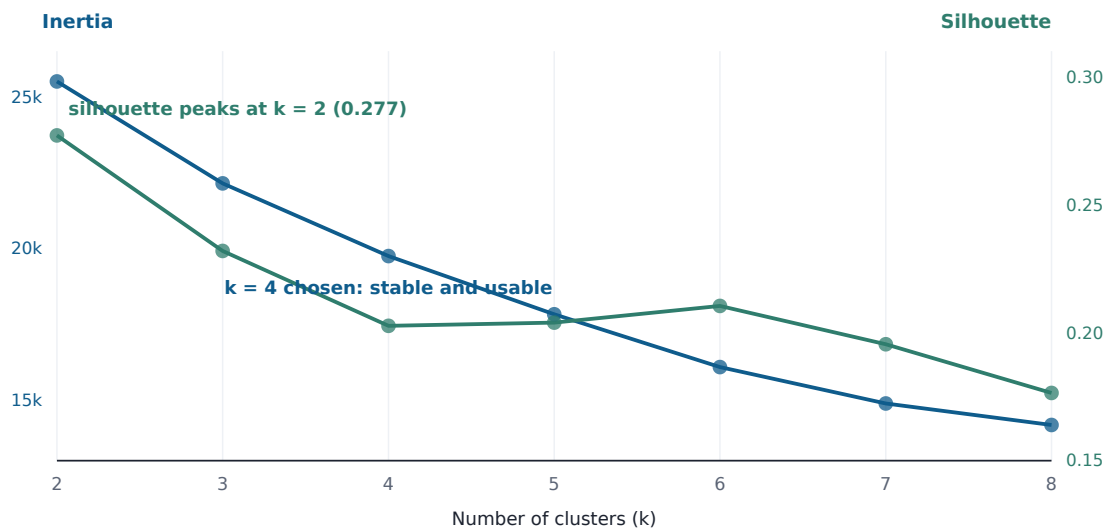


Figure 38.1 – Choosing k on Riverstone's accounts. Neither curve gives a clean answer: the elbow is a gentle curve, and the silhouette prefers the least useful answer, $k = 2$.

Reading it, plainly. There's no elbow worth the name: inertia slides down smoothly. The silhouette is highest at $k = 2$ (0.277) and never rises again. Taken literally, the maths says "two clusters", and even that score is modest; a silhouette below about 0.3 means the groups overlap heavily.

That's the truth about most business data. Customers don't fall into neat, separated balls; they spread out along a continuum of size and activity, and clustering **cuts** that continuum rather than discovering gaps in it. The cuts can still be useful. So the honest way to report this work is: "these groups are a convenient division of a continuous customer base, not a discovery of natural types."

Stability: would slightly different data give the same groups?

A more practical test than the silhouette: run the clustering again with different random starts, and on random subsets of the rows. If the groups survive, they're a stable description; if they scatter, they were an accident. **Adjusted Rand index (ARI)** compares two groupings of the same rows regardless of label names: 1 means identical, 0 means no more agreement than chance.

```

from sklearn.metrics import adjusted_rand_score

base = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(X)
for seed in [1, 2, 3]:
    other = KMeans(n_clusters=4, n_init=10, random_state=seed).fit_predict(X)
    print(
        f"seed {seed}: agreement with the first run"
        f" (adjusted Rand) {adjusted_rand_score(base, other):.3f}"
    )

rng = np.random.default_rng(38)
for trial in range(3):
    rows = rng.choice(len(X), size=int(0.8 * len(X)), replace=False)
    subset = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(X[rows])
    print(
        f"80% sample {trial + 1}: agreement with the same rows in the full run "
        f"{adjusted_rand_score(base[rows], subset):.3f}"
    )

```

```

seed 1: agreement with the first run (adjusted Rand) 0.996
seed 2: agreement with the first run (adjusted Rand) 0.997
seed 3: agreement with the first run (adjusted Rand) 0.994
80% sample 1: agreement with the same rows in the full run 0.991
80% sample 2: agreement with the same rows in the full run 0.984
80% sample 3: agreement with the same rows in the full run 0.969

```

Reading it. Agreement across random starts is 0.994–0.997, and across 80% samples 0.969–0.991. At $k = 4$, the division is highly reproducible even though the silhouette is unimpressive. Those two facts sit together comfortably: the boundaries land in the same places every time, they are drawn through a crowd rather than through empty space.

The business test

The last question is the one that decides: **can someone act differently for each group?** Four groups that map to four sales approaches beat two groups that score better and mean nothing. Riverstone's sales team can handle four; that, plus the stability above, is why the rest of this chapter uses $k = 4$.

38.3 Profiling and naming clusters

A cluster number means nothing until it's described. Profile every cluster on the features used, on features left out, and on size and value:

```

accounts["cluster"] = base
profile = accounts.groupby("cluster").agg(
    accounts=("account_id", "size"),
    revenue_2024=("revenue_2024", "median"),
    orders=("orders_2024", "median"),
    categories=("categories_bought", "mean"),
    discount_pct=("avg_discount_pct", "mean"),
    late_days=("late_payment_days", "mean"),
    days_since_order=("days_since_last_order", "median"),
    tenure=("tenure_months", "median"),
    churn_rate=("churned_2025", "mean"),
)
print(profile.round(2).to_string())
print()
print(
    (pd.crosstab(accounts["cluster"], accounts["segment"], normalize="index") * 100)
    .round(0)
    .to_string()
)
print()
print("share of 2024 revenue by cluster:")
print(
    (
        accounts.groupby("cluster")["revenue_2024"].sum()
        / accounts["revenue_2024"].sum()
        * 100
    )
    .round(1)
    .to_string()
)

```

	accounts	revenue_2024	orders	categories	discount_pct	late_days	days_since_order
tenure	churn_rate						
cluster							
0	1088	282400.0	13.0	3.21	5.63	11.40	23.0
26.0	0.03						
1	944	1083800.0	48.0	2.39	8.56	11.99	8.0
27.0	0.03						
2	2398	145600.0	7.0	1.28	4.50	16.34	35.0
26.0	0.08						
3	570	73450.0	2.0	1.48	4.16	17.10	228.5
24.0	0.41						

segment	Hospitality	Retail	Wholesale
cluster			
0	35.0	52.0	12.0
1	14.0	16.0	69.0
2	38.0	53.0	9.0
3	37.0	62.0	2.0

share of 2024 revenue by cluster:

cluster	
0	15.8
1	62.0
2	19.9
3	2.3

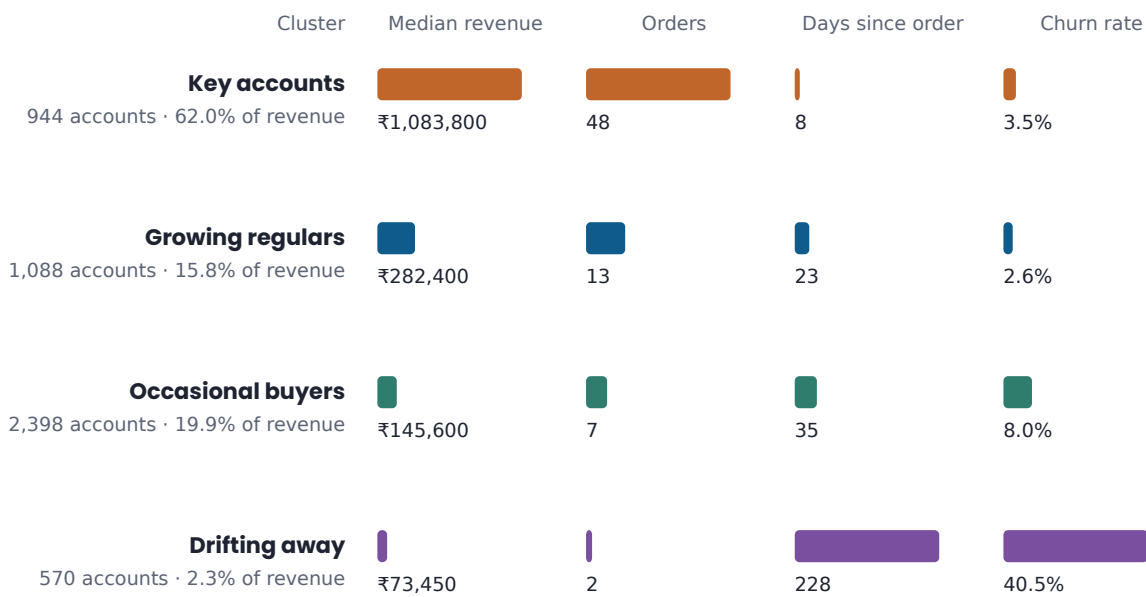


Figure 38.2 — The four clusters, described by the numbers that separate them. Cluster sizes and revenue shares are very different: the smallest cluster earns the most.

Naming them is the step that turns a model into something a sales team uses:

Cluster	Name	Size	Median revenue	What defines it	What to do
1	Key accounts	944	₹1,083,800	48 orders a year, deepest discounts, mostly wholesale (69%), ordered days ago	Named account managers; protect the relationship; review discounts
0	Growing regulars	1,088	₹282,400	13 orders, buys 3.2 of 4 categories, healthy recency	Cross-sell the fourth category; the most obvious upgrade path
2	Occasional buyers	2,398	₹145,600	7 orders, 1.3 categories, the largest group	Low-touch: catalog emails, self-service reordering
3	Drifting away	570	₹73,450	2 orders, last order 228 days ago, 41% churned in 2025	Win-back calls, or accept and stop spending on them

Three things worth noticing.

- **Value is concentrated.** The 944 key accounts are 19% of the customer base and **62%** of 2024 revenue. The 570 drifting accounts are 2.3%. If effort followed head count instead of value, it would be badly misplaced.
- **Segment labels and behavior differ.** Cluster 1 is 69% wholesale, but every cluster contains all three segments. "Wholesale" describes what a customer sells; the cluster describes how they buy.
- **Churn wasn't used** to build the clusters, yet the churn rate runs from 2.6% to 41%. That's an external check: the groups line up with something they were never shown. Section 38.8 measures it.

Watch out: clusters are a snapshot, not a category. Accounts move between clusters as they order more or go quiet. Re-run the clustering on a schedule, and expect the sales team to ask why a customer "changed type". Chapter 39's stability ideas and Chapter 52's monitoring apply here too.

38.4 Hierarchical clustering

How it works

Hierarchical clustering doesn't ask for k up front. **Agglomerative** clustering starts with every row as its own cluster and repeatedly merges the two closest, until everything is one cluster. The record of merges is a **dendrogram**, a tree you can cut at any height to get any number of clusters.

"Closest" needs a rule, the **linkage**:

- **Ward** (the usual default) merges the pair that increases total within-cluster variance least. It tends to give compact, similar-sized clusters.
- **Complete** linkage uses the distance between the two furthest members; **average** uses the mean distance; **single** uses the closest pair, which can chain long straggly clusters together.

The cost is memory and time: it compares every pair, so it grows with the square of the number of rows. Beyond a few thousand rows, cluster a sample.

```
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage

sample_rows = np.random.default_rng(38).choice(len(X), size=800, replace=False)
tree = linkage(X[sample_rows], method="ward")
for k in [2, 3, 4, 5]:
    labels = fcluster(tree, t=k, criterion="maxclust")
    print(
        f"cut into {k}: sizes {np.bincount(labels)[1:]}    "
        f"agreement with k-means {adjusted_rand_score(base[sample_rows], labels):.3f}"
    )
```

```
cut into 2: sizes [147 653]    agreement with k-means 0.256
cut into 3: sizes [147 287 366]    agreement with k-means 0.357
cut into 4: sizes [147 287 50 316]    agreement with k-means 0.413
cut into 5: sizes [147 119 168 50 316]    agreement with k-means 0.470
```

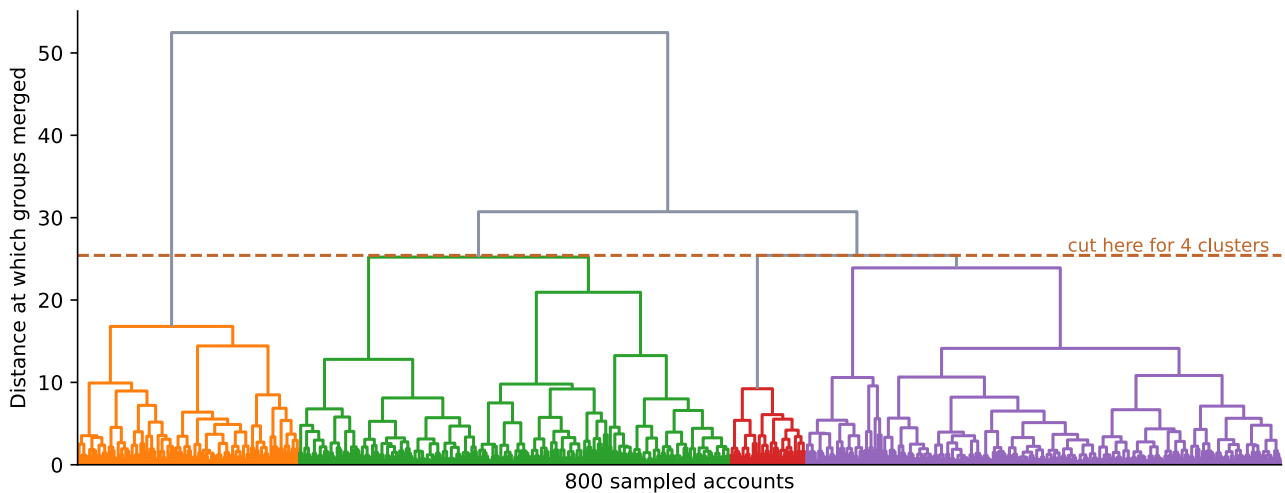


Figure 38.3 – A dendrogram of 800 accounts (Ward linkage). Each merge is drawn at the height that measures how different the merged groups were; cutting across the tree at a chosen height gives that many clusters.

Reading it. Cut into four, the tree gives clusters of 147, 287, 50, and 316 accounts, and it agrees with the k-means grouping to an ARI of 0.413: related, but far from the same. The two methods cut a continuous cloud in different places, another reminder that these groups are conveniences rather than discoveries. Hierarchical clustering earns its place when you want the tree itself: a picture of which groups sit inside which, which product ranges or store types nest together.

38.5 DBSCAN

How it works

DBSCAN (density-based spatial clustering of applications with noise) doesn't take k . It grows clusters from **dense** regions:

- A row is a **core point** if at least `min_samples` rows lie within a radius `eps` of it.
- Core points that are within `eps` of each other join the same cluster, and non-core rows within reach are attached at the edge.
- Everything else is labeled **noise** (`-1`), and isn't in any cluster.

Two properties follow. DBSCAN finds clusters of **any shape**, including long curved ones that defeat k-means, and it **refuses to classify** rows in sparse regions, which is useful when you want only the well-defined groups.

```
from sklearn.cluster import DBSCAN

print(" eps   clusters   noise points")
for eps in [0.5, 0.8, 1.0, 1.2, 1.6]:
    labels = DBSCAN(eps=eps, min_samples=10).fit_predict(X)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    print(f"{eps:>4}   {n_clusters:>8}   {(labels == -1).sum():>12}")
```

eps	clusters	noise points
0.5	2	4,678
0.8	7	1,971
1.0	1	854
1.2	1	313
1.6	1	47

Reading it. Every value of `eps` gives an unsatisfying answer. Small radii (0.5) leave 4,678 of 5,000 accounts as noise; large radii (1.2 and up) swallow almost everything into one cluster. There's no setting in between that gives several balanced clusters, and the reason is section 38.2's: this cloud has no genuine gaps for DBSCAN to find. **On this data, DBSCAN's honest answer is that there are no density-separated groups.**

That's worth taking seriously rather than tuning away. DBSCAN shines on spatial data (delivery points, sensor readings, GPS traces), where dense regions and empty space really exist, and it doubles as an anomaly detector: the noise points are the odd ones out.

Key settings: `eps` (the radius; sweep it as above) and `min_samples` (a common starting point is twice the number of features).

38.6 Seeing many dimensions: PCA, t-SNE, UMAP

Seven features can't be drawn. Three methods squeeze them into two dimensions for a picture:

- **PCA** (Chapter 35) finds the straight-line directions of greatest variance. It's fast, deterministic, reversible, and its axes have meaning through their loadings. It can't unfold curved structure.
- **t-SNE** places rows so that each row's **near neighbours** stay near, allowing the map to bend. It's excellent at showing local groupings, slow on large data, and its global layout (distances between blobs, blob sizes) is not meaningful.
- **UMAP** does something similar, usually faster, and preserves a little more of the global structure. Both have random starts and settings (`perplexity` , `n_neighbors`) that change the picture.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap

pca = PCA().fit(X)
print("PCA share of variance:", pca.explained_variance_ratio_[:4].round(3))
print(
    "first two components together:", f"{pca.explained_variance_ratio_[:2].sum():.1%}"
)
coords_pca = pca.transform(X)[: , :2]

plot_rows = np.random.default_rng(38).choice(len(X), size=1500, replace=False)
coords_tsne = TSNE(n_components=2, perplexity=30, random_state=38).fit_transform(
    X[plot_rows]
)
coords_umap = umap.UMAP(n_components=2, random_state=38).fit_transform(X[plot_rows])
print(f"t-SNE on {len(plot_rows):,} accounts, output shape {coords_tsne.shape}")
print("UMAP output shape:", coords_umap.shape)
```

```
PCA share of variance: [0.397 0.143 0.136 0.121]
first two components together: 54.1%
t-SNE on 1,500 accounts, output shape (1500, 2)
UMAP output shape: (1500, 2)
```

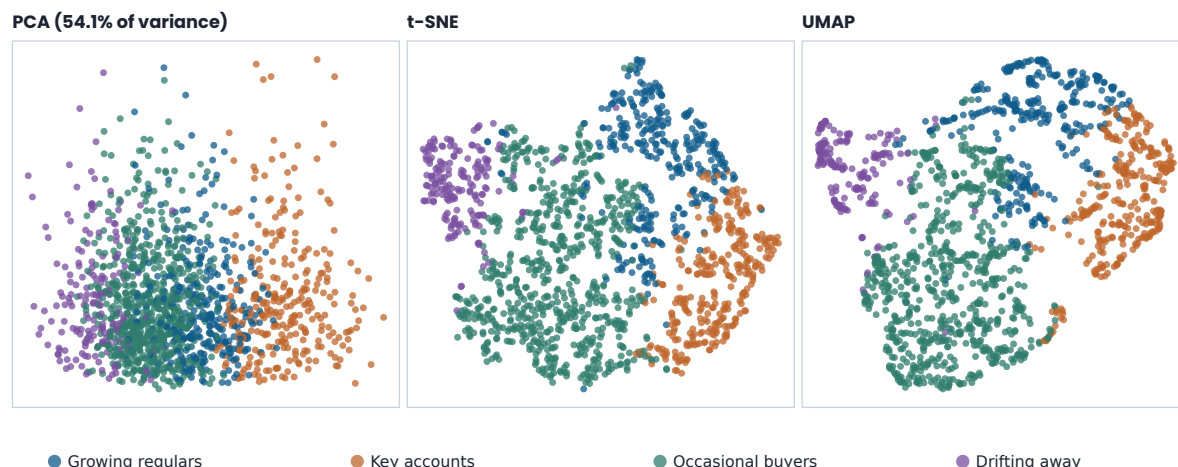


Figure 38.4 — The same accounts under PCA, t-SNE, and UMAP, colored by their *k*-means cluster. The clusters hold together in all three, and in all three they're touching, not separated: the same continuum the silhouette reported.

Reading it. PCA's first two components hold 54.1% of the variance, so the flat picture is missing nearly half the information. t-SNE and UMAP produce prettier, more separated pictures of the same 1,500 accounts, and that separation is partly a property of the methods, not of Riverstone's customers.

Watch out: don't over-read a t-SNE or UMAP picture. The distance between two blobs, the size of a blob, and the empty space between them carry little meaning; those methods are free to stretch and shrink regions. Never feed t-SNE coordinates into a predictive model as features, and never present one as evidence that groups are "clearly separated". Use them to explore and to illustrate, then verify with numbers.

38.7 Anomaly detection

Isolation forests

An **isolation forest** finds rows that are quick to isolate. It builds many random trees: at each step it picks a random feature and a random split point. Ordinary rows sit in crowded regions and need many splits to be separated; unusual rows get cut off after a few. The average number of splits needed becomes the **anomaly score**.

It's fast, it handles many features, and it needs no labels. The `contamination` setting says what share of rows to flag.

```

from sklearn.ensemble import IsolationForest

forest = IsolationForest(contamination=0.02, random_state=38).fit(X)
accounts["outlier_score"] = forest.decision_function(X) # lower = stranger
flagged = accounts.nsmallest(100, "outlier_score")
print(
    f"100 strangest accounts: churn rate {flagged['churned_2025'].mean():.1%} "
    f"vs {accounts['churned_2025'].mean():.1%} overall"
)
columns = [
    "segment",
    "revenue_2024",
    "orders_2024",
    "avg_discount_pct",
    "late_payment_days",
    "days_since_last_order",
    "churned_2025",
]
print(accounts.nsmallest(5, "outlier_score")[columns].to_string(index=False))

```

```

100 strangest accounts: churn rate 21.0% vs 9.7% overall
  segment  revenue_2024  orders_2024  avg_discount_pct  late_payment_days  days_since_last_order
churned_2025
Wholesale    7966300.0         77         11.7         40.0             7.0
0
Wholesale    1551400.0         82          9.0         45.0             3.0
0
  Retail      94600.0          2          2.6         63.0          365.0
1
Wholesale    4152200.0         62          6.9         32.0            10.0
0
Wholesale    3042000.0         68          7.0         55.0             3.0
1

```

Reading it. Among the 100 strangest accounts, 21% churned in 2025, against 9.7% overall. The flagged accounts fall into two kinds: enormous wholesale accounts that are unusual because nothing else is that large, and tiny accounts that ordered once long ago. Both are worth a human look, for opposite reasons.

That points at how anomaly detection is used in practice: **as a review queue, not a decision**. Nobody should be dropped because a model called them unusual. Typical uses are fraud triage (Chapter 39's cost thinking applies), data quality checks (an account with a 60% discount is probably a data-entry error), and sensor monitoring (Chapter 40 does this over time).

Other tools for the same job: `LocalOutlierFactor` (compares a row's local density with its neighbours'), `OneClassSVM` (learns a boundary around normal data), and DBSCAN's noise points. For anomalies over time, forecasting residuals work better (Chapter 40).

38.8 Judging unsupervised results

With no target, there are four kinds of evidence, in rising order of usefulness:

1. **Internal measures** (silhouette, Davies–Bouldin, inertia) score how compact and separated the clusters are, using only the features. They're comparable between runs on the same data, and they never prove that groups are real.
2. **Stability** (section 38.2) asks whether the same structure appears on different samples and seeds. Low stability is fatal; high stability is reassuring.
3. **External checks** compare the groups with something they were never given.
4. **Business usefulness**: can each group be described in a sentence and treated differently, and does acting on it change a number the company cares about? That's the one that matters, and it usually needs an experiment (Chapter 30).

The external check here: the clusters were built without any churn information. How much do they know about it?

```
from sklearn.metrics import roc_auc_score

rates = accounts.groupby("cluster")["churned_2025"].mean()
scored = accounts["cluster"].map(rates)
print("churn rate by cluster:", rates.round(3).to_dict())
auc = roc_auc_score(accounts["churned_2025"], scored)
print(f"using the cluster alone to rank churn risk: AUC {auc:.3f}")
print("(Chapter 37's supervised models scored 0.80 to 0.85 on this target.)")
```

```
churn rate by cluster: {0: 0.026, 1: 0.035, 2: 0.08, 3: 0.405}
using the cluster alone to rank churn risk: AUC 0.759
(Chapter 37's supervised models scored 0.80 to 0.85 on this target.)
```

Reading it. Ranking accounts by their cluster's churn rate gives an AUC of 0.759, against 0.80–0.85 for the supervised models in Chapter 37 that were trained on churn directly. Four unlabelled groups recover most of the ranking power of a model built for the job. That's a genuine external check, and also a caution: **when you have a target, use it.** Clustering isn't a substitute for a supervised model; it's what you reach for when there's no target at all, or when you need groups people can hold in their heads.

38.9 Market basket analysis

The question and the vocabulary

"Which products sell together?" The data is Riverstone's 2025 order lines: one row per product per order. A **basket** is one order, and the analysis counts how often sets of products appear in the same basket.

```

lines = pd.read_csv("../baskets/order_lines.csv")
products = pd.read_csv("../baskets/products.csv")
lines = lines.merge(
    products[["product_id", "product_name", "category"]], on="product_id"
)
lines = lines.merge(accounts[["account_id", "segment"]], on="account_id")
print(
    f"{lines['order_id'].nunique():,} orders, {len(lines):,} order lines, "
    f"{lines['product_name'].nunique():,} products, {lines['account_id'].nunique():,} accounts"
)
basket_sizes = lines.groupby("order_id").size()
print(
    "products per order: median",
    int(basket_sizes.median()),
    " mean",
    round(basket_sizes.mean(), 2),
    " max",
    int(basket_sizes.max()),
)
print(lines["product_name"].value_counts().head(5).to_string())

```

```

33,931 orders, 92,359 order lines, 24 products, 4,516 accounts
products per order: median 2 mean 2.72 max 10
product_name
Airtight Seal Pack      7286
Crate Lid (50L)         6500
Crate Lid (80L)         6313
Stacking Bin Large      5261
Drum Tap Fitting        5260

```

Three measures describe a rule such as *Storage Crate 50L* → *Crate Lid (50L)*:

support = share of all baskets containing the items (how common the pattern is)

confidence = share of baskets containing the "if" item that also contain the "then" item

lift = confidence ÷ the "then" item's own support (how much more likely than chance)

Lift is the key one. Lift of 1 means the two are independent; 3 means they appear together three times as often as chance would predict; below 1 means they repel each other.

Worked by hand

Of 33,931 orders, 4,170 contain a Storage Crate 50L, 6,500 contain a Crate Lid (50L), and 2,785 contain both:

$\text{support}(\text{crate}) = 4,170 \div 33,931 = 0.1229$

$\text{support}(\text{lid}) = 6,500 \div 33,931 = 0.1916$

$\text{support}(\text{both}) = 2,785 \div 33,931 = 0.0821$

$\text{confidence}(\text{crate} \rightarrow \text{lid}) = 0.0821 \div 0.1229 = \mathbf{0.668}$

$\text{lift} = 0.668 \div 0.1916 = \mathbf{3.49}$

Two thirds of crate orders include the matching lid, and that's 3.5 times the rate for orders in general.

```
baskets = lines.groupby("order_id")["product_name"].apply(set)
n_orders = len(baskets)
crate = baskets.apply(lambda s: "Storage Crate 50L" in s)
lid = baskets.apply(lambda s: "Crate Lid (50L)" in s)

support_a = crate.mean()
support_b = lid.mean()
support_ab = (crate & lid).mean()
confidence = support_ab / support_a
lift = confidence / support_b
print(f"orders: {n_orders:,}")
print(f"support(Storage Crate 50L)           {support_a:.4f}  ({crate.sum():,} orders)")
print(f"support(Crate Lid 50L)               {support_b:.4f}  ({lid.sum():,} orders)")
print(
    f"support(both)                          {support_ab:.4f}  ({(crate & lid).sum():,} orders)"
)
print(f"confidence(Crate 50L -> Lid 50L)      {confidence:.4f}")
print(f"lift                                    {lift:.2f}")
```

```
orders: 33,931
support(Storage Crate 50L)      0.1229  (4,170 orders)
support(Crate Lid 50L)         0.1916  (6,500 orders)
support(both)                  0.0821  (2,785 orders)
confidence(Crate 50L -> Lid 50L) 0.6679
lift                           3.49
```

Apriori: finding the rules automatically

Checking every possible rule is expensive: 24 products give 276 pairs and thousands of larger sets. The **Apriori** algorithm prunes the search with one observation: *if a set of items is rare, every larger set containing it is at least as rare*. So it finds items above a minimum support, then pairs, then triples, never looking at extensions of sets that already failed.

```
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

encoder = TransactionEncoder()
matrix = pd.DataFrame(encoder.fit_transform(baskets.tolist()), columns=encoder.columns_)
frequent = apriori(matrix, min_support=0.01, use_colnames=True)
rules = association_rules(frequent, metric="lift", min_threshold=1.2)
rules["if"] = rules["antecedents"].apply(lambda s: ", ".join(sorted(s)))
rules["then"] = rules["consequents"].apply(lambda s: ", ".join(sorted(s)))
print(f"{len(frequent)} frequent item sets, {len(rules)} rules with lift above 1.2")
top = rules.sort_values("lift", ascending=False).head(8)
print(
    top[["if", "then", "support", "confidence", "lift"]].round(3).to_string(index=False)
)
```

confidence	lift	if	then	support
0.555	8.899	Garden Table	Garden Chair	0.032
0.513	8.899	Garden Chair	Garden Table	0.032
0.347	5.559	Chair Cushion Set	Garden Chair	0.022
0.356	5.559	Garden Chair	Chair Cushion Set	0.022
0.482	4.790	Drum Tap Fitting, Industrial Crate 200L	Drum 60L	0.012
0.116	4.790	Drum 60L	Drum Tap Fitting, Industrial Crate 200L	0.012
0.075	4.667	Drum Tap Fitting	Drum 60L, Industrial Crate 200L	0.012
0.723	4.667	Drum 60L, Industrial Crate 200L	Drum Tap Fitting	0.012

How it works: `TransactionEncoder` turns baskets into a table of true/false columns, one per product. `apriori` finds every item set above `min_support` (1% of baskets here, about 340 orders). `association_rules` turns those sets into if-then rules and scores them, keeping rules with lift above 1.2.

Reading it. The strongest rules are the planted ones and they're the ones a warehouse manager would recognize: garden chairs with garden tables (lift 8.9), chairs with cushions (5.6), drums with tap fittings, crates with lids. Rules go in both directions with the same lift but different confidence: 55.5% of table buyers add chairs, while 51.3% of chair buyers add a table, because chairs are more common overall.

The confidence trap

Confidence alone is misleading, because a very common product has high confidence with everything:

```
common = "Crate Lid (50L)"
has_common = baskets.apply(lambda s: common in s)
for item in ["Chopping Board", "Serving Tray", "Folding Stool"]:
    has_item = baskets.apply(lambda s, i=item: i in s)
    both = (has_item & has_common).mean()
    conf = both / has_item.mean()
    print(
        f"{item:<16} -> {common}: support {both:.4f}    confidence {conf:.3f}    "
        f"lift {conf / has_common.mean():.2f}"
    )
```

Chopping Board	-> Crate Lid (50L): support 0.0160	confidence 0.142	lift 0.74
Serving Tray	-> Crate Lid (50L): support 0.0176	confidence 0.152	lift 0.79
Folding Stool	-> Crate Lid (50L): support 0.0064	confidence 0.142	lift 0.74

Reading it. A Crate Lid (50L) appears in 14% of baskets that contain a chopping board, which sounds like a pattern. Its lift is **0.74**: a chopping-board basket is *less* likely than average to contain a crate lid. The lid is common enough (19% of all baskets) that 14% is below chance. Always read confidence next to lift and support.

Watch out: three more basket traps. (1) **Trivial rules:** "buys crate lid → buys crate" may be a packaging rule, not an insight. (2) **Support too low:** a rule holding in 12 of 33,931 baskets can have enormous lift and no business value; that's why `min_support` exists. (3) **Correlation isn't causation:** a rule says the two appear together, not that pushing one sells the other. Testing a bundle offer with an experiment (Chapter 30) is what proves it.

Turning rules into money

A rule earns its keep when it points at a specific action:

```
drum_orders = set(lines.loc[lines["product_name"] == "Drum 60L", "order_id"])
tap_orders = set(lines.loc[lines["product_name"] == "Drum Tap Fitting", "order_id"])
missing = drum_orders - tap_orders
missing_accounts = lines.loc[lines["order_id"].isin(missing), "account_id"].nunique()
print(f"orders with Drum 60L: {len(drum_orders):,}")
share = len(missing) / len(drum_orders)
print(f"of those, without a Drum Tap Fitting: {len(missing):,} ({share:.1%})")
print(f"accounts placing at least one such order: {missing_accounts:,}")
segment_mix = (
    lines[lines["order_id"].isin(missing)].groupby("segment")["order_id"].nunique()
)
print("those orders by segment:", segment_mix.to_dict())
```

```
orders with Drum 60L: 3,412
of those, without a Drum Tap Fitting: 1,001 (29.3%)
accounts placing at least one such order: 613
those orders by segment: {'Hospitality': 35, 'Retail': 69, 'Wholesale': 897}
```

What to tell the sales head. "Two thirds of orders containing a 50-litre crate already include the matching lid, and drums almost always come with a tap fitting. But 1,001 orders across 613 accounts, nearly all wholesale, included a 60-litre drum with no tap fitting. That's a checkout prompt, a pre-filled line on the order form, or a call from the account manager, and we can measure whether it works by trying it with half the accounts first."

Chapter 42 builds the next step: a recommender that scores products for each customer rather than working from global rules.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Clustering unscaled features	Clusters differ only by the biggest-unit column	Standardize inside a pipeline (Chapter 36)
Leaving skewed money columns raw	A handful of huge accounts dominate every distance	Log-transform first
Including the target in the features	"Clusters" that are the target in disguise	Cluster on descriptive features only; use the target as an external check

Mistake	Symptom	Fix
Choosing k by silhouette alone	$k = 2$ every time, and useless groups	Weigh elbow, silhouette, stability, and whether the business can act on the groups
Reporting clusters as discovered types	"Our customers fall into four natural groups"	Say "a convenient division of a continuum" when the silhouette is low
Comparing cluster numbers between runs	"Cluster 2 moved to cluster 0"	Compare groupings with the adjusted Rand index
No stability check	Groups change every time the data updates	Re-run with different seeds and on subsamples
Reading distances in a t-SNE or UMAP picture	"These two groups are far apart, so they're very different"	Treat those layouts as sketches; verify with numbers
Using t-SNE coordinates as model features	Unreproducible, unexplainable models	Use PCA if you need components as features
Hierarchical clustering on huge data	Memory errors or a very long wait	Cluster a sample, or use k-means
Tuning DBSCAN until it gives k clusters	Either everything is noise or everything is one cluster	Accept that there may be no density-separated groups
Treating anomalies as decisions	Customers cut off because a model called them odd	Use anomaly scores as a review queue
Judging rules by confidence	"Everything leads to our best-selling product"	Read lift and support alongside confidence
Acting on very rare rules	A bundle nobody buys	Set a sensible <code>min_support</code> for the business
Assuming a rule is causal	A bundle launch that changes nothing	Test it (Chapter 30)

In the real world: the four segments that already existed

In May 2026, Anita Rao asks Meera for "proper customer segments" before the annual sales planning meeting. The marketing agency Riverstone used two years ago delivered nine segments with names like *Urban Aspirants*; nobody ever used them.

Meera builds the clustering in this chapter and gets four groups. Before presenting anything, she does three checks.

Does it survive re-running? Different seeds and 80% samples give adjusted Rand scores above 0.96. The groups are stable.

Does it know anything it wasn't told? She never showed it churn. Cluster 3's churn rate is 41%, against 2.6% for cluster 1. It does.

Does it match how the team already thinks? This is the check that changes the meeting. She takes the four profiles to the three sales executives, without the cluster numbers, and asks how they'd describe their customers. Neha describes "my regulars", "the big wholesale accounts I babysit", "the ones who order once a quarter", and "the ones who stopped answering". Four groups, near enough the same four.

At the planning meeting, Meera doesn't present a model. She presents a table: four groups, what each is worth, how many accounts each contains, and which of the three executives owns most of each. The headline is the concentration: **19% of accounts produce 62% of revenue, and the 570 drifting accounts produce 2.3%.**

Anita's question is the right one: "So *what changes?*" Three things. Key accounts get quarterly reviews instead of ad-hoc contact. Growing regulars get the cross-sell campaign for their missing product category, which the basket rules in section 38.9 can fill in. And the drifting accounts get one scripted win-back call each, after which they stop receiving sales attention, which frees roughly a day a week across the team.

Vikram, skeptical, asks why the earlier agency's nine segments failed. Meera's answer: "*They were built from demographics we don't act on, and nobody could tell which segment a customer was in without asking the agency. Ours come from our own order data, we can re-run them every quarter, and any rep can work out a customer's group from three numbers.*"

Six months later, two of the four groups have been renamed by the sales team, which Meera counts as the clearest sign the work landed.

Tools

- **scikit-learn** (tested on 1.8.0; current release at the time of writing 1.9.1): `KMeans` , `AgglomerativeClustering` , `DBSCAN` , `silhouette_score` , `adjusted_rand_score` , `PCA` , `TSNE` , `IsolationForest` . `MiniBatchKMeans` handles millions of rows.
 - **SciPy** for `linkage` , `dendrogram` , and `fcluster` , which give the tree that scikit-learn's version doesn't draw.
 - **umap-learn** 0.5.12 (`pip install umap-learn`) for UMAP.
 - **mlxtend** 0.25.0 (`pip install mlxtend`) for `TransactionEncoder` , `apriori` , and `association_rules` . For very large baskets, the FP-Growth implementation in the same library is faster.
 - **In SQL**: basket counts are a self-join of order lines on `order_id` with a `product_a < product_b` filter, which is often how a first pass is done on data too large to bring into pandas (Chapter 13's patterns).
 - Everything ran on one CPU core, Python 3.12.3, on 18 September 2026. The slowest step is t-SNE on 1,500 accounts, a few seconds.
 - **Companion files**: `companion/generate_riverstone_accounts.py` (Chapter 37) and `companion/generate_riverstone_baskets.py` (seed 20238) build the two datasets. Run the chapter's code from `companion/ch38/` . Data spec: `planning/data/riverstone-baskets.md` .
-

The project: customer segmentation and a product-bundle analysis

Goal: a segmentation a sales team could use, and a short list of product bundles worth testing, both defended with evidence rather than a chart.

Option A: your own data. Any customer, product, or transaction table you can use. Remove personal details first.

Option B: Riverstone. `accounts.csv` and the basket files.

Steps:

1. **Choose features deliberately.** List every column and say why it's in or out. Exclude anything that is a target or an outcome.
2. **Prepare.** Log-transform skewed money columns, handle blanks, and scale.
3. **Cluster** with k-means for $k = 2$ to 8. Record inertia and silhouette, and plot both.
4. **Check stability** with at least three seeds and three 80% subsamples, reported as adjusted Rand scores.
5. **Choose k** and justify it in three sentences: what the numbers say, what the business can act on, and what you're giving up.
6. **Profile and name** each cluster, with size, value share, and two or three defining numbers. Every name must be a phrase a sales rep would use.
7. **Cross-check** with one method you didn't use (hierarchical or DBSCAN) and one external column the clustering never saw.
8. **Draw one picture** (PCA, and optionally t-SNE or UMAP), with a caption that says what the picture does *not* prove.
9. **Basket analysis:** run Apriori, keep rules above a support and lift you justify, and remove trivial ones.
10. **Write a one-page recommendation:** the groups, what changes for each, two bundles to test, and how you'd measure whether any of it worked.

Stretch goals:

- Cluster on **behavior over time** instead of totals: build monthly order counts per account and cluster the 12-number sequences.
- Try **RFM** (recency, frequency, monetary value), the classic retail segmentation, and compare it with your clusters using the adjusted Rand index.
- Use **Gaussian mixture models** (`GaussianMixture`), which give each account a probability of belonging to each group instead of a hard label, and compare with k-means.
- Run basket analysis **per segment** and see whether the rules differ between wholesale and hospitality baskets.

You've got it when...

- ☐ I can explain the two steps of k-means and run them by hand on a handful of rows.
 - ☐ I always scale (and log skewed money) before any distance-based method.
 - ☐ I choose k using the elbow, the silhouette, stability, and business usefulness together, and I say when the data has no natural groups.
 - ☐ I compare two clusterings with the adjusted Rand index rather than by cluster number.
 - ☐ I profile and name clusters so that someone who has never seen the model can use them.
 - ☐ I can read a dendrogram and say what cutting it at a given height does.
 - ☐ I know what DBSCAN's noise label means and when density-based clustering is the right tool.
 - ☐ I can explain why t-SNE and UMAP pictures shouldn't be read for distances, and why PCA components can be used as features but t-SNE coordinates shouldn't.
 - ☐ I can use an isolation forest to build a review queue, and I never let it make the decision.
 - ☐ I can calculate support, confidence, and lift by hand, and explain why a high-confidence rule can still be worthless.
 - ☐ I judge an unsupervised result by stability, an external check, and whether anything changes because of it.
-

Recap

- **Unsupervised learning** finds structure without a target, and has no test set to prove it right.
 - **k-means** alternates assigning rows to the nearest **centroid** and moving centroids to their members' average, minimizing **inertia**. Scale first; set `n_init`; labels are arbitrary.
 - Choose k with the **elbow**, the **silhouette**, **stability** (adjusted Rand across seeds and subsamples), and above all whether the business can act on the groups. On continuous data, expect low silhouettes and say so.
 - **Profile and name** clusters with size, value share, and defining numbers.
 - **Hierarchical clustering** builds a **dendrogram** you can cut at any height; Ward linkage is the usual default; it doesn't scale to large data.
 - **DBSCAN** grows clusters from dense regions and labels the rest **noise**; it finds any shape and refuses to invent groups where there are none.
 - **PCA** is fast and meaningful; **t-SNE** and **UMAP** show local neighborhoods but not global distances, and are for pictures, not features.
 - **Isolation forests** score how easily a row is separated: a review queue for odd accounts, fraud triage, and data errors.
 - **Market basket analysis** measures **support**, **confidence**, and **lift**; **Apriori** finds rules efficiently; read lift and support alongside confidence, and test a bundle before believing it.
 - Judge unsupervised work by **stability**, an **external check**, and whether a decision changes.
-

Practice exercises

Code exercises run from `companion/ch38/` after the chapter's code (they use `accounts`, `X`, `FEATURES`, `base`, `lines`, `baskets`, `matrix`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) Two centroids sit at (0, 0) and (4, 4). Which cluster does the point (3, 1) join, and what is its squared distance to that centroid?
2. (*hand*) A cluster has 3 members at (2, 2), (4, 6), and (6, 4). Where is its centroid, and what is the cluster's contribution to inertia?
3. (*hand*) Of 1,000 baskets, 200 contain tea, 300 contain sugar, and 90 contain both. Calculate support, confidence, and lift for *tea* → *sugar*. Is the pattern worth reporting?
4. Which method fits each job: (a) group delivery points into service zones of any shape; (b) split 5,000 customers into four groups for four sales approaches; (c) show 20 features on one chart; (d) find 50 accounts worth a manual data check?

Core

5. Re-run k-means on the same accounts **without scaling** the features. Profile the clusters and describe what happened.
6. Cluster with $k = 5$ and compare it with the $k = 4$ grouping using the adjusted Rand index. Which cluster split in two?
7. Add `complaints_2024` to the feature list and re-run k-means with $k = 4$. Does the grouping change (adjusted Rand against `base`), and do the profiles change?
8. Compute the silhouette score for each of the four clusters separately (use `silhouette_samples`). Which cluster is best defined, and which overlaps most?
9. Run Apriori with `min_support=0.005` and `min_threshold=1.5`, and count how many rules you get. How many involve three or more products?
10. For each of the four clusters, find the top three products by number of baskets (join the order lines to `accounts`). Do the clusters buy differently?

Stretch

11. Fit a `GaussianMixture` with four components on `X`, compare its labels with `base` (adjusted Rand), and report how many accounts have a top probability below 0.6.
12. Build monthly order counts per account from the basket data, cluster those 12-number profiles with $k = 4$, and describe the shapes you find.
13. Run market basket analysis separately for wholesale and hospitality baskets. Name one rule that appears in one and not the other, and explain it.

Think about it

14. A colleague clusters customers using revenue, orders, units, and order value. The clusters come out ordered purely by size. What happened, and what would you change?

- l5. The sales head asks: "Are these groups real?" Answer in three sentences, using this chapter's evidence.
- l6. A basket rule says *printer* → *printer paper* with lift 6. Marketing wants to promote paper to printer buyers. What's the flaw, and what would you propose instead?
-

Key terms

unsupervised learning · clustering · k-means · centroid · inertia · assignment step · update step · `n_init` · elbow method · silhouette score · adjusted Rand index (ARI) · stability · cluster profiling · hierarchical clustering · agglomerative · dendrogram · linkage (Ward, complete, average, single) · cutting the tree · DBSCAN · density · core point · `eps` · `min_samples` · noise points · dimensionality reduction · PCA · explained variance · t-SNE · perplexity · UMAP · local versus global structure · anomaly detection · isolation forest · contamination · local outlier factor · market basket analysis · basket (transaction) · item set · support · confidence · lift · Apriori · FP-Growth · trivial rule · external validation · Gaussian mixture model · RFM

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, takes the churn and lead models and turns their scores into decisions with costs, and covers the fairness checks that segmentation work also needs.
 - **Chapter 40, Time Series & Forecasting**, detects anomalies over time, where an isolation forest on static features can't help.
 - **Chapter 41, NLP Foundations**, clusters support tickets by topic, using the same k-means on text features.
 - **Chapter 42, Recommender Systems & Ranking**, goes beyond global basket rules to per-customer recommendations, reusing this chapter's basket data.
 - **Chapter 30, Experiments**, is how a bundle or a win-back campaign is proved to work.
 - **Chapters 55 and 58** cluster embeddings of documents, which is this chapter's k-means on a different kind of feature.
 - **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers k-means, choosing *k*, DBSCAN versus k-means, PCA versus t-SNE, and "how do you know your clusters are any good?", which is the question most candidates answer badly.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Squared distance to (0, 0) is $3^2 + 1^2 = 10$. To (4, 4) it is $(3 - 4)^2 + (1 - 4)^2 = 1 + 9 = 10$. It's an exact **tie**, which is the point of the question: the row sits on the boundary, and the algorithm has to break the tie by a rule (scikit-learn takes the lower cluster number). Rows near boundaries have silhouette scores near 0 and can flip between runs, which is why individual assignments in an overlapping cloud shouldn't be treated as facts about a customer.

2. Centroid = $((2 + 4 + 6) \div 3, (2 + 6 + 4) \div 3) = (4, 4)$. Squared distances: (2, 2) gives $4 + 4 = 8$; (4, 6) gives $0 + 4 = 4$; (6, 4) gives $4 + 0 = 4$. The cluster contributes **16** to inertia.

3. $\text{support}(\text{tea}) = 200 \div 1,000 = 0.20$; $\text{support}(\text{sugar}) = 0.30$; $\text{support}(\text{both}) = 90 \div 1,000 = 0.09$. $\text{confidence}(\text{tea} \rightarrow \text{sugar}) = 0.09 \div 0.20 = 0.45$. $\text{lift} = 0.45 \div 0.30 = 1.5$. Worth reporting: the support is high enough to matter (9% of all baskets) and tea buyers take sugar 1.5 times as often as basket buyers in general. Whether it's *actionable* is another question, since both are staples people may buy together out of habit.

4. (a) **DBSCAN**: service zones follow roads and rivers, so clusters have irregular shapes, and outlying addresses should be left unassigned. (b) **k-means**: a fixed number of similar-sized groups, which is exactly what it produces. (c) **PCA** first, because its axes mean something and it's reproducible; UMAP as a second picture if the structure looks curved. (d) **Isolation forest**: it ranks rows by how unusual they are, so you can take the 50 strangest.

5.

```
unscaled = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(
    accounts[FEATURES]
)
print(accounts.groupby(unscaled)[FEATURES].median().round(1).to_string())
print("\ncluster sizes:", np.bincount(unscaled).tolist())
print(
    "agreement with the scaled clustering:",
    round(adjusted_rand_score(base, unscaled), 3),
)
```

	log_revenue	orders_2024	categories_bought	avg_discount_pct	late_payment_days
0	12.7	15.0	2.0	5.9	11.0
13.0	27.0				
1	11.4	4.0	1.0	4.4	14.0
166.5	25.0				
2	11.2	2.0	1.0	4.1	14.5
332.0	25.0				
3	11.9	6.0	1.0	4.6	13.0
68.0	25.0				

cluster sizes: [3014, 468, 244, 1274]
agreement with the scaled clustering: 0.135

Without scaling, the clusters are almost entirely bands of `days_since_last_order`, with `orders_2024` a distant second: those two columns have the largest raw spread (about 79 and 17), while `log_revenue` varies by about 1 and `categories_bought` by about 1. The other five features barely influence the result, and the agreement with the scaled clustering is poor. This is Chapter 35's units lesson in its most expensive form.

6.

```
five = KMeans(n_clusters=5, n_init=10, random_state=38).fit_predict(X)
print("adjusted Rand, k=4 vs k=5:", round(adjusted_rand_score(base, five), 3))
print(pd.crosstab(base, five).to_string())
```

```
adjusted Rand, k=4 vs k=5: 0.723
col_0    0    1    2    3    4
row_0
0         0    0  107  944   37
1        892    0   26   22    4
2         0    0  521    3  1874
3         0  518   33    0   19
```

Read the table by rows: each $k = 4$ cluster should map mostly to one $k = 5$ cluster, except the one that split. The extra cluster comes mainly out of the largest group, the occasional buyers, which splits by recency and order count. That's the usual pattern when a continuum is cut more finely: the biggest slice divides rather than a new type appearing.

7.

```
with_complaints = FEATURES + ["complaints_2024"]
X2 = StandardScaler().fit_transform(accounts[with_complaints])
labels2 = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(X2)
print(
    "adjusted Rand vs the original clustering:",
    round(adjusted_rand_score(base, labels2), 3),
)
shown = ["revenue_2024", "orders_2024", "complaints_2024", "days_since_last_order"]
print(accounts.groupby(labels2)[shown].median().round(1).to_string())
```

```
adjusted Rand vs the original clustering: 0.897
revenue_2024  orders_2024  complaints_2024  days_since_last_order
0      1092000.0         48.0             1.0                8.0
1      298200.0         14.0             0.0               22.0
2       73300.0          2.0             0.0             227.0
3      144200.0          7.0             0.0             36.0
```

The grouping barely changes. Complaints rise with order volume, so the new feature mostly repeats information the clustering already had through `orders_2024`. Adding features that duplicate existing ones quietly doubles their weight in the distance calculation; adding new information is what changes a clustering.

8.

```

from sklearn.metrics import silhouette_samples

scores = silhouette_samples(X, base)
print(
    pd.DataFrame({"cluster": base, "silhouette": scores})
      .groupby("cluster")["silhouette"]
      .agg(["mean", "size"])
      .round(3)
      .to_string()
)
print("accounts with a negative silhouette:", int((scores < 0).sum()))

```

```

      mean  size
cluster
0      0.174 1088
1      0.205  944
2      0.206 2398
3      0.234  570
accounts with a negative silhouette: 126

```

The drifting cluster is the best defined (0.234), because "no order for months" separates it cleanly from everything else. The growing regulars score lowest (0.174): they sit between the key accounts and the occasional buyers, with neighbours on both sides. Every cluster's score is low, which is section 38.2's message again. The accounts with negative silhouettes sit closer to a neighboring cluster's members than to their own, and would move if the data shifted slightly.

9.

```

frequent_small = apriori(matrix, min_support=0.005, use_colnames=True)
rules_small = association_rules(frequent_small, metric="lift", min_threshold=1.5)
sizes = rules_small["antecedents"].apply(len) + rules_small["consequents"].apply(len)
print(f"{len(frequent_small)} item sets, {len(rules_small)} rules with lift above 1.5")
print("rules by number of products:", sizes.value_counts().sort_index().to_dict())
print(
    f"smallest support among those rules: {rules_small['support'].min():.4f} "
    f"({rules_small['support'].min() * len(baskets):.0f} orders)"
)

```

```

338 item sets, 464 rules with lift above 1.5
rules by number of products: {2: 32, 3: 394, 4: 38}
smallest support among those rules: 0.0050 (170 orders)

```

Halving the support threshold produces many more rules, most of them involving three or more products and most of them variations on the same planted pairs. More rules is not more insight: the useful output of a basket analysis is usually a handful of rules a person can act on, which is why you raise `min_support` until the list fits on one page.

10.

```

cluster_of = accounts.set_index("account_id")["cluster"]
lines["cluster"] = lines["account_id"].map(cluster_of)
top_products = (
    lines.dropna(subset=["cluster"])
    .groupby(["cluster", "product_name"])["order_id"]
    .nunique()
    .rename("orders")
    .reset_index()
)
for cluster, group in top_products.groupby("cluster"):
    best = group.nlargest(3, "orders")
    print(
        f"cluster {int(cluster)}: "
        + ", ".join(f"{r.product_name} ({r.orders:,})" for r in best.itertuples())
    )

```

```

cluster 0: Airtight Seal Pack (1,816), Crate Lid (50L) (1,186), Crate Lid (80L) (1,135)
cluster 1: Drum Tap Fitting (4,325), Dolly Wheels (set) (4,268), Crate Lid (50L) (3,814)
cluster 2: Airtight Seal Pack (2,159), Crate Lid (50L) (1,383), Crate Lid (80L) (1,279)
cluster 3: Airtight Seal Pack (192), Crate Lid (80L) (129), Serving Tray (119)

```

The lists overlap heavily, because the most common products are common everywhere. Differences show up in what follows the top three, and they match the segment mix of each cluster: the key-accounts cluster, mostly wholesale, leans on industrial items, while the occasional buyers' baskets are storage and kitchen. For sharper differences, compare each cluster's share of a product against the overall share rather than raw counts.

11.

```

from sklearn.mixture import GaussianMixture

mixture = GaussianMixture(n_components=4, random_state=38).fit(X)
soft = mixture.predict_proba(X)
labels_gmm = soft.argmax(axis=1)
print("adjusted Rand vs k-means:", round(adjusted_rand_score(base, labels_gmm), 3))
print(
    "accounts whose best group has probability below 0.6:",
    int((soft.max(axis=1) < 0.6).sum()),
)

```

```

adjusted Rand vs k-means: 0.394
accounts whose best group has probability below 0.6: 206

```

A Gaussian mixture allows stretched, tilted clusters instead of round ones, so its groups differ noticeably from k-means. The interesting output is the uncertainty: 206 accounts, about 4%, have no confident home, which is the same message as the low silhouette, stated more usefully. Soft memberships are worth the extra complexity when you want to treat borderline customers carefully rather than forcing them into a group.

12.

```

lines["month"] = pd.to_datetime(lines["order_date"]).dt.month
monthly = (
    lines.groupby(["account_id", "month"])["order_id"]
    .nunique()
    .unstack(fill_value=0)
    .reindex(columns=range(1, 13), fill_value=0)
)
monthly = monthly[monthly.sum(axis=1) >= 6]
shapes = StandardScaler().fit_transform(monthly.div(monthly.sum(axis=1), axis=0))
shape_labels = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(shapes)
print(f"{len(monthly):,} accounts with at least 6 orders")
print(
    pd.DataFrame(
        monthly.div(monthly.sum(axis=1), axis=0).to_numpy(), index=shape_labels
    )
    .groupby(level=0)
    .mean()
    .round(3)
    .to_string()
)

```

```

1,727 accounts with at least 6 orders

```

	0	1	2	3	4	5	6	7	8	9	10	11
0	0.079	0.053	0.081	0.098	0.059	0.118	0.091	0.055	0.073	0.170	0.058	0.065
1	0.098	0.035	0.106	0.081	0.164	0.059	0.076	0.052	0.078	0.053	0.101	0.097
2	0.075	0.042	0.084	0.065	0.052	0.090	0.104	0.187	0.092	0.056	0.077	0.076
3	0.085	0.164	0.074	0.071	0.058	0.058	0.075	0.072	0.084	0.063	0.098	0.099

Each row here is a *shape*, the share of a customer's orders falling in each month, not a level, so big and small accounts can share a pattern. The four groups differ mainly in which months are busiest. Each group peaks in a different month, which looks like seasonality until you remember that this generator spreads orders evenly across the year: with 12 noisy numbers per account, k-means will always find groups whose averages peak somewhere. That is the honest answer: clustering finds groups whether or not the data contains any. Chapter 40 handles seasonality properly, with data that has it.

13.

```

for segment in ["Wholesale", "Hospitality"]:
    segment_baskets = (
        lines[lines["segment"] == segment]
        .groupby("order_id")["product_name"]
        .apply(set)
    )
    encoded = TransactionEncoder()
    grid = pd.DataFrame(
        encoded.fit_transform(segment_baskets.tolist()), columns=encoded.columns_
    )
    seg_rules = association_rules(
        apriori(grid, min_support=0.01, use_colnames=True),
        metric="lift",
        min_threshold=1.5,
    )
    seg_rules["if"] = seg_rules["antecedents"].apply(lambda s: ", ".join(sorted(s)))
    seg_rules["then"] = seg_rules["consequents"].apply(lambda s: ", ".join(sorted(s)))
    best = seg_rules.nlargest(3, "lift")[
        ["if", "then", "support", "confidence", "lift"]
    ]
    print(f"--- {segment} ({len(segment_baskets):,} baskets)")
    print(best.round(3).to_string(index=False))

```

```

--- Wholesale (16,574 baskets)
      if                                then  support  confidence  lift
      Garden Chair                     Garden Table  0.014      0.476 18.531
      Garden Table                     Garden Chair  0.014      0.542 18.531
Drum Tap Fitting, Storage Crate 50L Crate Lid (50L), Drum 60L  0.010      0.313  9.493
--- Hospitality (7,905 baskets)
      if                                then  support  confidence  lift
Airtight Seal Pack, Garden Chair                     Garden Table  0.015      0.540 7.370
Garden Table Airtight Seal Pack, Garden Chair  0.015      0.211 7.370
Garden Chair                     Garden Table  0.042      0.530 7.242

```

The pairs that are planted in the data (chairs with tables, drums with taps, crates with lids) appear in both segments, because the rules were built into the products, not the customers. What differs is which rules **clear the support threshold**, and how strong they look: the chair-and-table pair has a lift of 18.5 in wholesale baskets against 7.2 in hospitality (furniture is rarer in wholesale orders, so co-occurrence stands out more), and the three-product industrial rules appear only in wholesale. That's the practical lesson: run basket analysis per segment when the segments buy different catalogues, or the rules of the largest segment will be the only ones you see.

14. Those four features all measure the same thing, the size of the account, and three of them are strongly correlated, so distance is dominated by size and the clusters come out as "big, medium, small". A useful segmentation needs features that vary **independently** of each other: keep one size measure, and add behavior (recency, product breadth, discount level, payment habits). The quick check is a correlation matrix of the features before clustering.

15. "They're stable: re-running with different random starts and on 80% samples gives almost identical groups. They know something they were never told: the groups' churn rates run from 3% to 41%, and churn was not among the features. But they aren't natural types, because the silhouette score is low and the accounts form a continuum rather than separated balls, so the boundaries are our choice of where to cut."

16. The rule is **trivial**: people who buy a printer need paper, so the association tells you nothing you didn't know, and the buyer probably already adds paper without prompting. Lift measures association, not the effect of an intervention, so it can't say whether the promotion changes behavior. Better proposals: target printer buyers who **didn't** buy paper (section 38.9's approach), promote paper at the right *time* (weeks after purchase, when the first ream runs out), or find the non-obvious partner products that also have high lift with printers. And whichever is chosen, test it on half the customers first (Chapter 30).